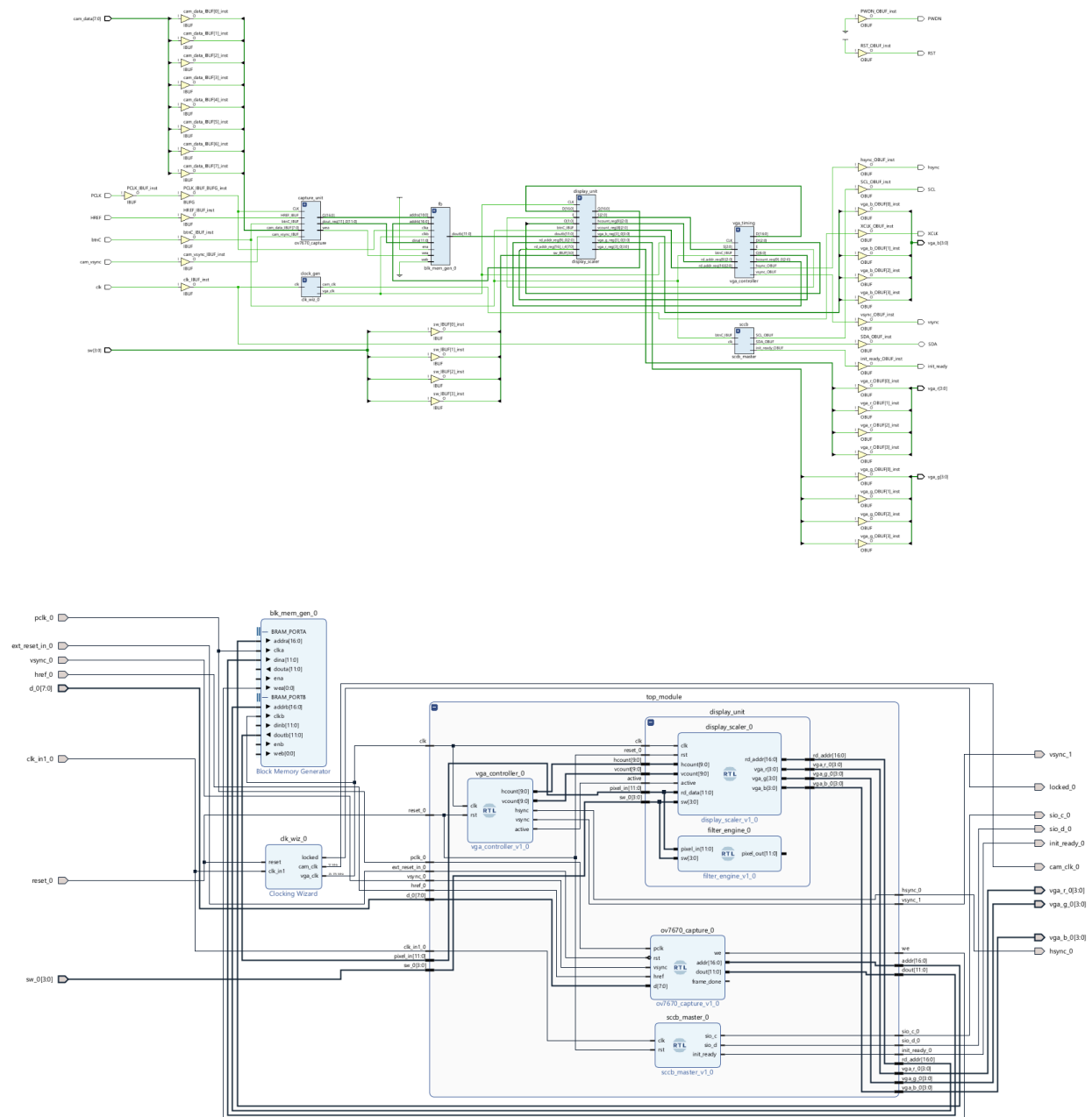


Group Member

Group name : **B2**

Name	Student Number
Kochakorn Srichay	6731301921
Navaporn Homjandee	6730265821
Napas Ruttanapunyagorn	6730262921

https://drive.google.com/drive/folders/1YELpIbwtBJ_UI7r69TViYd_zvh0rxerV?usp=sharing



Design Decision.

1. Module Architecture

1.1 Why did you structure your design this way?

1. Modular Design : We divide each system into independent modules to simplify development and debugging.
2. True Dual-Port RAM : We use BRAM because they provide two interfaces. One port writes using the PCLK and the other reads using VGA Clock.
3. Resource balancing : We use Downsampling technique and applied RGB444 color reduction.

1.2 What are the advantages of using this approach? (e.g., modularity, efficiency, ease of debugging)

4. The Camera Capture Unit only needs to write data because it captures pixel data from the camera and stores in buffer
5. The VGA Controller only needs to read data because it fetches the stored pixels to display them on the monitor.
6. Since each module operates on a separate port with its own clock, there are no access conflicts between the camera and the display. This makes True Dual Port RAM the ideal choice for handling asynchronous data flow while preventing screen tearing.

2. Communication Protocols

2.1 Why did you choose this specific protocol (e.g., SPI, I2C, UART,AXI) for communication?

1. SCCB for Camera Configuration : Because It is the strict hardware standard required to set up the camera. Using a low frequency and pull-up resistors on its two wires ensures stable data transmission.
2. Native Interface for Memory Transfer : We choose over AXI to save FPGA resources and avoid complex logic. It provides the exact, predictable timing needed for VGA synchronization and is much easier to debug since the camera only writes and the display only reads.

2.2 How does this protocol suit your design requirements?

We use SCCB protocol because the OV7670 camera strictly uses it for initializing the register. It can operate at 100kHz because the system can only send commands once during startup. And it is easy to implement to control state without consuming the board hardware resource.

3. Clock and Data Transfer Rate

3.1 Why did you select this specific clock frequency?

Because the system clock uses 100MHz which required for internal FSMs and SCCB interface but Pixel Clock for VGA uses 25.175MHz and OV7670 also uses 24MHz as Camera XCLK for driving

3.2 How does the data rate impact system performance?

Its impact in two areas. First, the image data (PCLK) requires a dual-port BRAM buffer and downsampling to lagging video. Second, the configuration data (SCCB) minimizes noise that ensure stable and accurate startup initialization

3.3 Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?

We address the constraint by using a Dual-Port BRAM to separate the clock domains to prevent data corruption and custom logic to compensate for BRAM read latency. For external device compatibility, color bits are converted and the image is downsampled to address the limited BRAM memory. And we minimize noise by enabling internal Pull-up resistors at SCL and SDA pins combined with a low 100 kHz configuration data rate.

4. Resource Utilization

4.1 How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?

LUTs : In grayscale processing, we use bit-shifting and addition to calculate average instead of using multiple or division. This can save logic resources while maintaining display performance.

FFs : We decided to implement FSM for SCCB configuration instead of using external IP cores. This minimizes Flip-Flop usage while ensuring the camera setup remains stable and efficient.

BRAMs : We captured image downsampled and reduced color depth to 12-bit before writing in BRAM to get a smooth realtime VGA display without exceeding the limited memory capacity of the board.

4.2 Why did you optimize certain parts of the design?

Because we have to manage limited hardware resources for the system requirements. For example

1. Memory Optimization

Since we downscaled the resolution to 320x240 and the color format was reduced from RGB565 to RGB444, this reduced the memory requirement from 3.7 Megabits which exceeds the 1.8 Megabits of Block RAM on the Artix-7 FPGA used in the Basys 3 Board.

2. Address Calculation

We used a simple bit-shifting operation ($\text{cam_x} \gg 1$ and $\text{cam_y} \gg 1$) to divide the pixel coordinates by two, which requires almost zero logic resources and calculates the exact memory addresses instantly with zero latency.

Implementation Detail

Module : top_module

Module code :

```
module top_module (
    input wire      clk,          // Main 100 MHz board clock
    input wire      btnC,        // Reset Button
    output wire     init_ready,  // LED : high when SCCB config is
finished
    input wire [3:0] sw,         // Filter Switches

    // Camera Hardware Interface
    input wire      PCLK,
    input wire      cam_vsync,
    input wire      HREF,
    input wire [7:0] cam_data,
    output wire     XCLK,
    output wire     RST,
    output wire     PWDN,
    output wire     SCL,
    inout wire      SDA,

    // VGA Hardware Interface
    output wire     hsync,
    output wire     vsync,
    output wire [3:0] vga_r,
    output wire [3:0] vga_g,
    output wire [3:0] vga_b
);

    wire      cam_clk;          // Clock for camera : 24 MHz
    wire      vga_clk;          // clock for VGA : 25.175 MHz
    wire      reset = btnC;

    // Clock generation : Create specific speeds for Camera and VGA
    clk_wiz_0 clock_gen (
        .clk(clk),
        .cam_clk(cam_clk),
        .vga_clk(vga_clk)
```

```

);

assign XCLK = cam_clk;          // Drive camera master clock
assign RST  = 1'b1;
assign PWDN = 1'b0;

// Camera Configuration : Sends register settings via SCCB
sccb_master sccb (
    .clk(clk),
    .rst(reset),
    .init_ready(init_ready),
    .sio_c(SCL),
    .sio_d(SDA)
);

wire [16:0] write_addr;          // 17 bits for 76800 addresses
wire [11:0] capture_data;        // 12 bits for RGB444
wire we;                          // write enable for BRAM

// Capture unit : Converts raw camera bytes into 12-bits RGB pixels
ov7670_capture capture_unit (
    .pclk(PCLK),
    .rst(reset),
    .vsync(cam_vsync),
    .href(HREF),
    .d(cam_data),
    .we(we),
    .addr(write_addr),
    .dout(capture_data)
);

wire [16:0] read_addr;           // 17 bits for 76800 addresses
wire [11:0] bram_data;           // 12 bits for RGB444

// Frame Buffer : True Dual-port memory to store one frame of video
// Port A : Camera Writes | Port B : VGA Reads
blk_mem_gen_0 fb (
    .clka(PCLK),
    .wea(we),

```

```

        .addra(write_addr),
        .dina(capture_data),
        .ena(1'b1),

        .clkb(vga_clk),
        .addrb(read_addr),
        .doutb( bram_data),
        .web  (1'b0)
    );

// VGA Control & Filter : Generates sync signals and applies filters
wire [9:0] hcount;
wire [9:0] vcount;
wire video_active;

vga_controller vga_timing (
    .clk(vga_clk),
    .rst(reset),
    .hcount(hcount),
    .vcount(vcount),
    .hsync(hsync),
    .vsync(vsync),
    .active(video_active)
);

display_scaler display_unit (
    .clk(vga_clk),
    .rst(reset),
    .hcount(hcount),
    .vcount(vcount),
    .active(video_active),
    .rd_data( bram_data),
    .rd_addr(read_addr),
    .sw(sw),
    .vga_r(vga_r),
    .vga_g(vga_g),
    .vga_b(vga_b)
);

```



```
endmodule
```

Module Description :

The top_module is the top-level for handling OV7670 camera and VGA display. It manages all sub-module interconnections, clock distribution, and hardware I/O to coordinate real-time pixel capturing, True Dual-Port RAM buffering, and image filtering.

Module : ov7670_capture

Module code :

```
module ov7670_capture (
    input wire      pclk,
    input wire      rst,
    input wire      vsync,
    input wire      href,
    input wire [7:0] d,

    output reg      we,
    output reg [16:0] addr,
    output reg [11:0] dout,
    output reg      frame_done
);

    reg [7:0] d_latched;    // Temporary register for first byte data
    reg byte_idx;           // byte index for clarify current data's
byte
    reg [9:0] cam_x;        // Horizontal coordinates of the current
address
    reg [8:0] cam_y;        // Vertical coordinates of the current
address

    always @( posedge pclk ) begin
        if (rst) begin
            we          <= 1'b0;
            addr         <= 17'd0;

```

```

        dout      <= 12'd0;
        frame_done <= 1'b0;
        byte_idx   <= 1'b0;
    end else begin
        we <= 1'b0;

        // Reset coordinates at the start of a new frame
        if (vsync) begin
            byte_idx <= 0; cam_x <= 0; cam_y <= 0; we <= 1'b0;
        end else if (href) begin
            // HREF is high when a valid row of pixels is being sent

            // First byte : store the value and wait for second byte
            if (!byte_idx) begin
                d_latched <= d;
                we <= 1'b0;
            end else begin
                // Second byte : combine into the first byte to be 12
bits (RGB444)
                // Only store into the BRAM on x and y is odd (to
downsampling from 640 * 480 to 320 * 240 pixels per frame)
                if (!cam_x[0] && !cam_y[0]) begin
                    addr <= ((cam_y >> 1) * 17'd320) + (cam_x >> 1);
                end
            // Calculate memory address
            dout <= {d_latched[7:4], d_latched[2:0], d[7],
d[4:1]}; // RRRRXGGG GXXBBBBX
            we <= 1'b1; // write enable to BRAM
            end else we <= 1'b0;
            cam_x <= cam_x + 1;
        end
        byte_idx <= !byte_idx; // Toggle byte counter
    end else begin
        byte_idx <= 0; we <= 1'b0;
        if (cam_x > 0) begin // End 1 horizontal line
            cam_x <= 0;
            cam_y <= cam_y + 1;
        end
    end
end
end

```

```
end  
endmodule
```

Module Description :

The ov7670_capture is the data acquisition module for handling incoming pixel data from the OV7670 camera. It manages spatial downsampling, color quantization, and clock synchronization to coordinate realtime pixel formatting, memory address calculation, and writing to the True Dual-Port RAM buffer.

Module : sccb_master

Module code :

```
module sccb_master (
    input  wire clk,          // Main 100MHz clock
    input  wire rst,          // System reset

    output reg  sio_c,        // SCCB Clock signal
    output reg  sio_d,        // SCCB Data signal

    output reg  init_ready    // High when all registers are configured
);

// Generates a 'tick' to slow down the 100MHz clock for SCCB timing
reg [8:0] divcnt;
reg tick;

always @( posedge clk ) begin
    if (rst) begin
        divcnt <= 0;
        tick   <= 0;
    end else begin
        tick <= 0;

        // Divide clock to reach ~400kHz
        if (divcnt == 249) begin
            divcnt <= 0;
            tick   <= 1;
        end else divcnt <= divcnt + 1;
    end
end

// Stores 16-bit values: [8-bit Register Address, 8-bit Data Value]
reg [15:0] rom_data;
reg [7:0]  rom_addr;

always @(*) begin
    case (rom_addr)

        // 1. Soft Reset : Resets all camera registers to default
```

```
0 : rom_data = 16'h1280;

// 2. Clocking & PLL : Sets internal camera speeds
1 : rom_data = 16'h1100;
2 : rom_data = 16'h6B4A;
3 : rom_data = 16'h3B0A;

// 3. Format & Scaling : Sets output to RGB and specific
resolutions
4 : rom_data = 16'h1204;
5 : rom_data = 16'h40D0;
6 : rom_data = 16'h3A04;

7 : rom_data = 16'h0C00;
8 : rom_data = 16'h3E00;

9 : rom_data = 16'h703A;
10 : rom_data = 16'h7135;
11 : rom_data = 16'h7211;
12 : rom_data = 16'h73F1;
13 : rom_data = 16'hA202;

// 4. Windowing : Defines the visible area of the sensor
14 : rom_data = 16'h1713;
15 : rom_data = 16'h1801;
16 : rom_data = 16'h32BF;

17 : rom_data = 16'h1902;
18 : rom_data = 16'h1A7A;
19 : rom_data = 16'h030A;

// 5. Color Matrix : Adjusts how colors are calculated
20 : rom_data = 16'h4F80;
21 : rom_data = 16'h5080;
22 : rom_data = 16'h5100;
23 : rom_data = 16'h5215;
24 : rom_data = 16'h539C;
25 : rom_data = 16'h54D4;
26 : rom_data = 16'h589E;
```

```

        // 6. AEC / AGC / AWB : Auto Exposure and White Balance
settings
    27 : rom_data = 16'h13EF;
module sccb_master (
    input wire clk,          // Main 100MHz clock
    input wire rst,          // System reset

    output reg sio_c,        // SCCB Clock signal
    output reg sio_d,        // SCCB Data signal

    output reg init_ready    // High when all registers are configured
);

// Generates a 'tick' to slow down the 100MHz clock for SCCB timing
reg [8:0] divcnt;
reg tick;

always @( posedge clk ) begin
    if (rst) begin
        divcnt <= 0;
        tick    <= 0;
    end else begin
        tick <= 0;

        // Divide clock to reach ~400kHz
        if (divcnt == 249) begin
            divcnt <= 0;
            tick    <= 1;
        end else divcnt <= divcnt + 1;
    end
end

// Stores 16-bit values: [8-bit Register Address, 8-bit Data Value]
reg [15:0] rom_data;
reg [7:0] rom_addr;

always @(*) begin
    case (rom_addr)

```

```

//
=====

// 1. Soft Reset
//
=====

// Reg 12 (COM7) = 0x80 -> Clear all registers to defaults
and clear FSMs
0 : rom_data = 16'h1280;

//
=====

// 2. Clocking & PLL (Internal Camera Speeds)
//
=====

// Reg 11 (CLKRC) = 0x00 -> Internal clock pre-scaler divide
by 1
1 : rom_data = 16'h1100;
// Reg 6B (DBLV) = 0x4A -> Enable PLL (x4 clock
multiplier), bypass regulator
2 : rom_data = 16'h6B4A;
// Reg 3B (COM11) = 0x0A -> Enable night mode, allow frame
rate to drop in low light
3 : rom_data = 16'h3B0A;

//
=====

// 3. Format & Scaling (Output Formats & Resolution)
//
=====

// Reg 12 (COM7) = 0x04 -> Change format to RGB output
(combines with Reg 40)
4 : rom_data = 16'h1204;
// Reg 40 (COM15) = 0xD0 -> Set output range 00-FF, Output
RGB565 format
5 : rom_data = 16'h40D0;
// Reg 3A (TSLB) = 0x04 -> Set UV line sequence,
auto-windowing configuration
6 : rom_data = 16'h3A04;
// Reg 0C (COM3) = 0x00 -> Single frame output option,

```

```

scale tracking disabled
    7 : rom_data = 16'h0C00;
    // Reg 3E (COM14) = 0x00 -> Disable manual pixel clock
scaling divider
    8 : rom_data = 16'h3E00;
    // Reg 70 (SCALING_XSC) = 0x3A -> Horizontal scaling test
pattern row calculation
    9 : rom_data = 16'h703A;
    // Reg 71 (SCALING_YSC) = 0x35 -> Vertical scaling test
pattern column calculation
    10 : rom_data = 16'h7135;
    // Reg 72 (SCALING_DCWCTR)= 0x11 -> Downsampling Control
(Downsample by 2)
    11 : rom_data = 16'h7211;
    // Reg 73 (SCALING_PCLK_DIV) = 0xF1 -> Divide Pixel Clock
(PCLK) by 2 for scaling
    12 : rom_data = 16'h73F1;
    // Reg A2 (SCALING_PCLK_DELAY)=0x02 -> Delay output pixel
clock to match data valid time
    13 : rom_data = 16'hA202;

    //
=====

    // 4. Windowing (Defines Visible Sensor Array Coordinates)
    //
=====

    // Reg 17 (HSTART) = 0x13 -> Horizontal sensor window start
position (high 8 bits)
    14 : rom_data = 16'h1713;
    // Reg 18 (HSTOP) = 0x01 -> Horizontal sensor window end
position (high 8 bits)
    15 : rom_data = 16'h1801;
    // Reg 32 (HREF) = 0xBF -> Horizontal edge control details
(fractional alignment bits)
    16 : rom_data = 16'h32BF;
    // Reg 19 (VSTART) = 0x02 -> Vertical sensor window start
position (high 8 bits)
    17 : rom_data = 16'h1902;
    // Reg 1A (VSTOP) = 0x7A -> Vertical sensor window end

```



```

position (high 8 bits)
    18 : rom_data = 16'h1A7A;
    // Reg 03 (VREF)    = 0x0A -> Vertical edge control details
(fractional alignment bits)
    19 : rom_data = 16'h030A;

    //
=====
    // 5. Color Matrix (RGB Calculation / Tint adjustments)
    //
=====
    // Reg 4F to 54 are color correction matrices (MTX1 to MTX6)
for true color representation
    20 : rom_data = 16'h4F80; // MTX1  - Red gain coefficient
matrix
    21 : rom_data = 16'h5080; // MTX2  - Green gain coefficient
matrix
    22 : rom_data = 16'h5100; // MTX3  - Blue gain coefficient
matrix
    23 : rom_data = 16'h5215; // MTX4  - Color space conversion
balance
    24 : rom_data = 16'h539C; // MTX5  - Color space conversion
balance
    25 : rom_data = 16'h54D4; // MTX6  - Color space conversion
balance
    // Reg 58 (MTXS)    = 0x9E -> Matrix sign control for color
adjustment algorithms
    26 : rom_data = 16'h589E;

    //
=====
    // 6. AEC / AGC / AWB (Auto Exposure, Auto Gain, Auto White
Balance)
    //
=====
    // Reg 13 (COM8)    = 0xEF -> Enable: Fast AGC, AEC, AWB, and
Fast Color Filtering
    27 : rom_data = 16'h13EF;
    // Reg 00 (GAIN)    = 0x00 -> Set initial hardware gain to 0dB

```

```

multiplier
    28 : rom_data = 16'h0000;
    // Reg 10 (AECH) = 0x00 -> Clear exposure value registers
to allow auto exposure control
    29 : rom_data = 16'h1000;
    // Reg 0D (COM4) = 0x40 -> Average window speed evaluation
for auto-exposure calculations
    30 : rom_data = 16'h0D40;
    // Reg 14 (COM9) = 0x38 -> Set automatic gain Ceiling limit
to 4x multiplier max
    31 : rom_data = 16'h1438;
    // Reg 24 (AEW) = 0x95 -> Upper limit threshold for Auto
Exposure luminance
    32 : rom_data = 16'h2495;
    // Reg 25 (AEB) = 0x33 -> Lower limit threshold for Auto
Exposure luminance
    33 : rom_data = 16'h2533;
    // Reg 26 (VPT) = 0xE3 -> Fast AGC/AEC target luminance
balance parameter
    34 : rom_data = 16'h26E3;

    //
=====

    // 7. Gamma Curve Adjustments (Non-linear
Brightness/Contrast)
    //
=====

    // Reg 7A to 89 define the 16 slope points of the gamma
distribution curve
    // used to translate raw linear sensor data into
eye-pleasing display colors.
    35 : rom_data = 16'h7A20; // SLOP - Gamma curve slope
point 1
    36 : rom_data = 16'h7B10; // GAM1 - Gamma curve point 2
    37 : rom_data = 16'h7C1E; // GAM2 - Gamma curve point 3
    38 : rom_data = 16'h7D35; // GAM3 - Gamma curve point 4
    39 : rom_data = 16'h7E5A; // GAM4 - Gamma curve point 5
    40 : rom_data = 16'h7F69; // GAM5 - Gamma curve point 6
    41 : rom_data = 16'h8076; // GAM6 - Gamma curve point 7

```

```

42 : rom_data = 16'h8180; // GAM7 - Gamma curve point 8
43 : rom_data = 16'h8288; // GAM8 - Gamma curve point 9
44 : rom_data = 16'h838F; // GAM9 - Gamma curve point 10
45 : rom_data = 16'h8496; // GAM10 - Gamma curve point 11
46 : rom_data = 16'h85A3; // GAM11 - Gamma curve point 12
47 : rom_data = 16'h86AF; // GAM12 - Gamma curve point 13
48 : rom_data = 16'h87C4; // GAM13 - Gamma curve point 14
49 : rom_data = 16'h88D7; // GAM14 - Gamma curve point 15
50 : rom_data = 16'h89E8; // GAM15 - Gamma curve point 16

//
=====
// 8. DSP & Denoise (Digital Processing Filter Engine)
//
=====

// Reg 41 (COM16) = 0x08 -> Enable Edge Enhancement
sharpening algorithms
51 : rom_data = 16'h4108;
// Reg 76 (OV_R45) = 0xE1 -> Active Color Matrix processing
and digital denoise active
52 : rom_data = 16'h76E1;
// Reg 33 (CHLF) = 0x0B -> Array low-pass color filter
offset adjustments
53 : rom_data = 16'h330B;
// Reg 3C (COM12) = 0x78 -> Enable noise-reduction windowing
offsets
54 : rom_data = 16'h3C78;
// Reg 69 (GFIX) = 0x00 -> Fix digital gain calculation
metrics to manual baseline
55 : rom_data = 16'h6900;
// Reg 74 (REG74) = 0x00 -> Disable digital gain boosts
(preserves clarity)
56 : rom_data = 16'h7400;
// Reg B0 (RSVD) = 0x84 -> Reserved register parameter
optimization for lens correction
57 : rom_data = 16'hB084;
// Reg B1 (ABLC1) = 0x00 -> Automatic Black Level
Calibration active
58 : rom_data = 16'hB100;

```

```

        // Reg B2 (RSVD)  = 0x0E -> Reserved optimization bits for
color separation thresholds
    59 : rom_data = 16'hB20E;
    // Reg B3 (THL_ST) = 0x82 -> Target threshold setup for dark
frame digital cleanup
    60 : rom_data = 16'hB382;

    //
=====

    // 9. Saturation & Contrast
    //
=====

    // Reg 67 (MANU)  = 0x80 -> Manual color saturation
initialization (U-channel)
    61 : rom_data = 16'h6780;
    // Reg 68 (MANV)  = 0x80 -> Manual color saturation
initialization (V-channel)
    62 : rom_data = 16'h6880;
    // Reg 56 (MANC)  = 0x40 -> Fixed manual contrast ratio
scaling index
    63 : rom_data = 16'h5640;

    //
=====

    // 10. Frame Stability & Sync Signals
    //
=====

    // Reg 15 (COM10) = 0x00 -> VSYNC changes on falling edge,
HREF normal polarity
    64 : rom_data = 16'h1500;
    // Reg 13 (COM8)  = 0xEF -> Re-assert Auto functions
(Ensures clock states haven't overwritten variables)
    65 : rom_data = 16'h13EF;
    // Reg 0E (COM5)  = 0x61 -> Drive settings optimized for
internal register reading stability
    66 : rom_data = 16'h0E61;
    // Reg 16 (RSVD)  = 0x00 -> Reserved sensor sync tuning
setup
    67 : rom_data = 16'h1600;

```

```

        // Reg 1E (MVFP) = 0x07 -> Enable mirror/flip on image data
output array if needed
        68 : rom_data = 16'h1E07;

        // End of configuration marker
        default : rom_data = 16'hFFFF;
    endcase
end

localparam WAIT = 0; // Wait between transmissions
localparam IDLE = 1; // Prepare to send next register
localparam START = 2; // SCCB Start Condition (Data pulls low before
Clock)
localparam SEND = 3; // Shift out 8 bits of data
localparam ACK = 4; // "Don't care" bit (acknowledge cycle)
localparam STOP = 5; // SCCB Stop Condition (Data pulls high after
Clock)
localparam NEXT = 6; // Move to next ROM address
localparam DONE = 7; // Configuration finished

reg [2:0] state;

reg [1:0] phase;
reg [2:0] bit_index;
reg [1:0] byte_index;
reg [7:0] current_byte;
reg [19:0] wait_cnt;

always @( posedge clk ) begin
    if (rst) begin
        state <= WAIT;
        sio_c <= 1;
        sio_d <= 1;
        rom_addr <= 0;
        init_ready <= 0;
        phase <= 0;
        bit_index <= 7;
        byte_index <= 0;
        current_byte <= 0;
    end
end

```

```

        wait_cnt <= 0;
    end else begin
        case (state)
            // Provides a delay for the camera to process data
            WAIT : begin
                if (wait_cnt == 1_000_000) begin
                    wait_cnt <= 0;
                    state <= IDLE;
                end else wait_cnt <= wait_cnt + 1;
            end
            IDLE : begin
                sio_c <= 1;
                sio_d <= 1;
                if (rom_data == 16'hFFFF) begin // Check for end of
ROM
                    init_ready <= 1;
                    state <= DONE;
                end else begin
                    current_byte <= 8'h42; // Camera Write ID
Address
                    byte_index <= 0;
                    bit_index <= 7;
                    phase <= 0;
                    state <= START;
                end
            end
        end
        // Start bit : SIO_D goes low while SIO_C is high
        START : begin
            if (tick) begin
                if (!phase) begin
                    sio_d <= 0;
                    phase <= 1;
                end else begin
                    sio_c <= 0;
                    phase <= 0;
                    state <= SEND;
                end
            end
        end
    end
end

```

```

// Transmits bits 7 down to 0
SEND : begin
    if (tick) begin
        if (!phase) begin
            sio_c <= 0;
            // Place bit on data line
            sio_d <= current_byte[bit_index];
            phase <= 1;
        end else begin
            // Pulse clock to signal data is ready
            sio_c <= 1;
            phase <= 0;
            if (bit_index == 0) state <= ACK; // Byte
finished
            else bit_index <= bit_index - 1;
        end
    end
end
// SCCB 3-phase write requires an ACK/Don't Care bit
ACK : begin
    if (tick) begin
        case (phase)
            0 : begin
                sio_c <= 0;
                sio_d <= 1;
                phase <= 1;
            end
            1 : begin
                sio_c <= 1;
                phase <= 2;
            end
            2 : begin
                sio_c <= 0;
                phase <= 0;
                if (byte_index == 2) state <= STOP; //
Sent ID, Addr, and Data
            else begin
                byte_index <= byte_index + 1;
                // Prepare next byte from ROM

```

```

                                case (byte_index)
                                    0: current_byte <=
rom_data[15:8];
                                    1: current_byte <=
rom_data[7:0];

                                endcase
                                bit_index <= 7;
                                state <= SEND;
                            end
                        end
                    endcase
                end
            end
        // Stop bit : SIO_D goes high while SIO_C is high
        STOP : begin
            if (tick) begin
                if (!phase) begin
                    sio_c <= 1;
                    sio_d <= 0;
                    phase <= 1;
                end else begin
                    sio_d <= 1;
                    phase <= 0;
                    state <= NEXT;
                end
            end
        end
    // Increment ROM address to configure next register
    NEXT : begin
        rom_addr <= rom_addr + 1;
        wait_cnt <= 0;
        state <= WAIT;
    end
    // All 68 registers sent successfully
    DONE : init_ready <= 1;
    default : state <= WAIT;
endcase
end
end

```



```
endmodule
```

Module Description :

The sccb_master is the configuration controller for handling the initial setup of the OV7670 camera. It manages the SCCB protocol timing and register data transmission to coordinate the realtime configuration of essential camera parameters such as resolution and RGB444 color formatting before the pixel capturing process begins.

Module : vga_controller

Module code :

```
module vga_controller (
    input wire      clk,
    input wire      rst,
    output reg [9:0] hcount,
    output reg [9:0] vcount,
    output wire      hsync,
    output wire      vsync,
    output wire      active
);

    reg reg_hsync;
    reg reg_vsync;
    reg reg_active;

    assign hsync = reg_hsync;
    assign vsync = reg_vsync;
    assign active = reg_active;

    always @( posedge clk ) begin
        if (rst) begin
            hcount <= 10'd0;
            vcount <= 10'd0;
        end else begin
            // Horizontal Counter: 0 to 799 (includes sync and blanking)
            if (hcount == 10'd799) begin
                hcount <= 10'd0;
            end
        end
    end
endmodule
```

```

        // Vertical Counter: 0 to 524
        if (vcount == 10'd524) vcount <= 10'd0;
        else vcount <= vcount + 10'd1;
    end else hcount <= hcount + 10'd1;

    // Generate HSync/VSync pulses based on industry standards
    reg_hsync <= (hcount < 10'd656) || (hcount >= 10'd752);
    reg_vsync <= (vcount < 10'd490) || (vcount >= 10'd492);

    // Determine if we are in the "Active" visible area
    reg_active <= (hcount < 10'd640) && (vcount < 10'd480);
end
end
endmodule

```

Module Description :

The vga_controller is the display management unit responsible for driving the VGA monitor. It manages Hsync and Vsync timing generation, screen coordinate tracking, and vga_clk synchronization to coordinate the real-time fetching and synchronized output of RGB444 pixel data from the True Dual-Port RAM buffer.

Module : display_scaler

Module code :

```

module display_scaler (
    input  wire      clk,
    input  wire      rst,

    input  wire [9:0] hcount,
    input  wire [9:0] vcount,
    input  wire      active,

    input  wire [11:0] rd_data,
    output reg [16:0] rd_addr,

    input  wire [3:0] sw,

```

```

output reg [3:0] vga_r,
output reg [3:0] vga_g,
output reg [3:0] vga_b
);

wire [8:0] fb_x;
wire [7:0] fb_y;
assign fb_x = hcount[9:1];
assign fb_y = vcount[8:1];

wire [16:0] addr_next;
assign addr_next = (fb_y * 17'd320) + fb_x;

reg active_d;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        rd_addr <= 17'd0;
        active_d <= 1'b0;

    end else begin
        if (active) rd_addr <= addr_next;
        active_d <= active;
    end
end

wire [11:0] pixel_display;

filter_engine filter (
    .pixel_in(rd_data),
    .sw(sw),
    .pixel_out(pixel_display)
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        vga_r <= 4'd0;
        vga_g <= 4'd0;

```

```

        vga_b <= 4'd0;
    end else begin
        if (active_d) begin
            vga_r <= pixel_display[11:8];
            vga_g <= pixel_display[7:4];
            vga_b <= pixel_display[3:0];
        end else begin
            vga_r <= 4'd0;
            vga_g <= 4'd0;
            vga_b <= 4'd0;
        end
    end
end
end
endmodule

```

Module Description :

The display_scaler is the resolution adaptation module for handling the upscaling of the 320x240 buffered image to fit a standard 640x480 VGA display. It manages from screen to memory coordinate mapping and pixel duplication to coordinate real-time image enlargement and precise read-address generation for fetching RGB444 data from the True Dual-Port RAM.

Module : filter_engine

Module code :

```

module filter_engine (
    input  wire [11:0] pixel_in, // input RGB444
    input  wire [3:0]  sw,       // Switches for filter
    output reg [11:0] pixel_out // output RGB444
);

    wire [3:0] r4 = pixel_in[11:8];
    wire [3:0] g4 = pixel_in[7:4];
    wire [3:0] b4 = pixel_in[3:0];

    wire [5:0] y_sum = r4 + g4 + b4;
    wire [3:0] gray  = y_sum[5:2];

```

```

wire [3:0] r_invt = 15 - r4;
wire [3:0] g_invt = 15 - g4;
wire [3:0] b_invt = 15 - b4;

always @(*) begin
    case (sw)
        4'b0000 : pixel_out = pixel_in;           // Normal
        4'b0001 : pixel_out = {gray, gray, gray}; // Grayscale
        4'b0010 : pixel_out = {4'b0000, 4'b0000, b4}; // Blue Only
        4'b0100 : pixel_out = {4'b0000, g4, 4'b0000}; // Green
Only
        4'b1000 : pixel_out = {r4, 4'b0000, 4'b0000}; // Red only
        4'b1111 : pixel_out = {r_invt, g_invt, b_invt}; // Inversion
        default : pixel_out = pixel_in;
    endcase
end
endmodule

```

Module Description :

The filter_engine is the image processing for handling realtime visual effects on the video stream. It manages RGB channel manipulation and hardware switch evaluation to coordinate the dynamic modification of RGB444 pixel data such as grayscale, color inversion, and color isolations before it is sent to the VGA display.

Extra Credit : Full VGA Resolution (640x480)

Design Solution : Reduced Color Depth

The Basys 3 board's FPGA offers a limited 1.8 Megabyte of Block RAM, making it impossible to store a full 640 * 480 pixels per frame at 12-bit color (RGB444), which demands roughly 4.6 Megabyte. To fit the frame buffer entirely within the on-chip memory and streamline the hardware design, we compressed the color depth to a 5-bit RGB221 format (2 bits Red, 2 bits Green, 1 bit Blue).

Implementation Detail

The provided modules are changed in detail from 320 * 240 pixels per frame implementation.

Module : top_module

Module code :

```
module top_module (
    input wire      clk,          // Main 100 MHz board clock
    input wire      btnC,         // Reset Button
    output wire     init_ready,   // LED : high when SCCB config is
finished
    input wire [3:0] sw,          // Filter Switches

    // Camera Hardware Interface
    input wire      PCLK,
    input wire      cam_vsync,
    input wire      HREF,
    input wire [7:0] cam_data,
    output wire     XCLK,
    output wire     RST,
    output wire     PWDN,
    output wire     SCL,
    inout wire      SDA,

    // VGA Hardware Interface
    output wire     hsync,
    output wire     vsync,
    output wire [3:0] vga_r,
```

```

output wire [3:0] vga_g,
output wire [3:0] vga_b
);

wire cam_clk; // Clock for camera : 24 MHz
wire vga_clk; // clock for VGA : 25.175 MHz
wire reset = btnC;
wire init_ready;

// Clock generation : Create specific speeds for Camera and VGA
clk_wiz_0 clock_gen (
    .clk(clk),
    .cam_clk(cam_clk),
    .vga_clk(vga_clk)
);

assign XCLK = cam_clk; // Drive camera master clock
assign RST = 1'b1;
assign PWDN = 1'b0;

// Camera Configuration : Sends register settings via SCCB
sccb_master sccb (
    .clk(clk),
    .rst(reset),
    .init_ready(init_ready),
    .sio_c(SCL),
    .sio_d(SDA)
);

wire [18:0] write_addr; // 19 bits for 307200 addresses
wire [4:0] capture_data; // 5 bits for RGB221
wire we; // write enable for BRAM

// Capture unit : Converts raw camera bytes into 12-bits RGB pixels
ov7670_capture capture_unit (
    .pclk(PCLK),
    .rst(reset),
    .vsync(cam_vsync),
    .href(HREF),

```

```

        .d(cam_data),
        .we(we),
        .addr(write_addr),
        .dout(capture_data)
    );

    wire [18:0] read_addr;          // 19 bits for 307200 addresses
    wire [4:0] bram_data;           // 5 bits for RGB221

    // Frame Buffer : True Dual-port memory to store one frame of video
    // Port A : Camera Writes | Port B : VGA Reads
    blk_mem_gen_0 fb (
        .clka(PCLK),
        .wea(we),
        .addra(write_addr),
        .dina(capture_data),
        .ena(1'b1),

        .clkb(vga_clk),
        .addrb(read_addr),
        .doutb(bram_data),
        .web (1'b0)
    );

    // VGA Control & Filter : Generates sync signals and applies filters
    wire [9:0] hcount;
    wire [9:0] vcount;
    wire video_active;

    vga_controller vga_timing (
        .clk(vga_clk),
        .rst(reset),
        .hcount(hcount),
        .vcount(vcount),
        .hsync(hsync),
        .vsync(vsync),
        .active(video_active)
    );

```



```

display_scaler display_unit (
    .clk(vga_clk),
    .rst(reset),
    .hcount(hcount),
    .vcount(vcount),
    .active(video_active),
    .rd_data(bram_data),
    .rd_addr(read_addr),
    .sw(sw),
    .vga_r(vga_r),
    .vga_g(vga_g),
    .vga_b(vga_b)
);

endmodule

```

Module Description : Since we are using 640 * 320 difference of data in the memory and difference size of color's bits, so the variable of addresses and color's bits are different.

Module : ov7670_capture

Module code :

```

module ov7670_capture (
    input wire    pclk,
    input wire    rst,
    input wire    vsync,
    input wire    href,
    input wire [7:0] d,

    output reg     we,
    output reg [18:0] addr,
    output reg [4:0] dout,
    output reg     frame_done
);

    reg [7:0] d_latched;    // Temporary register for first byte data
    reg byte_idx;           // byte index for clarify current data's

```

```

byte
    reg [9:0] cam_x = 0;    // Horizontal coordinates of the current
address
    reg [9:0] cam_y = 0;    // Vertical coordinates of the current
address

    always @( posedge pclk ) begin
        if (rst) begin
            we <= 1'b0;
            cam_x <= 0;
            cam_y <= 0;
            dout <= 12'd0;
            frame_done <= 1'b0;
            byte_idx <= 1'b0;
        end else begin
            we <= 1'b0;

            // Reset coordinates at the start of a new frame
            if (vsync) begin
                byte_idx <= 0; cam_x <= 0; cam_y <= 0; we <= 1'b0;
            end else if (href) begin
                // HREF is high when a valid row of pixels is being sent

                // First byte : store the value and wait for second byte
                if (!byte_idx) begin
                    d_latched <= d;
                    we <= 1'b0;
                end else begin
                    // Second byte : combine into the first byte to be 5
bits (RGB221)
                    addr <= (cam_y * 19'd640) + cam_x; // Calculate
memory address
                    dout <= {d_latched[7:6], d_latched[2:1], d[4]}; //
RRXXXGGX XXXBXXXX
                    we <= 1'b1; // write enable to BRAM
                    cam_x <= cam_x + 1;
                end
                byte_idx <= !byte_idx; // Toggle byte counter
            end else begin

```

```

        byte_idx <= 0; we <= 1'b0;
        if (cam_x > 0) begin // End 1 horizontal line
            cam_x <= 0;
            cam_y <= cam_y + 1;
        end
    end
end
end
endmodule

```

Module Description : Because we store difference size of data than the 320 * 240 pixels per frame, therefore the mapping of the color bits and address are calculate differently.

Module : display_scaler

Module code :

```

module display_scaler (
    input  wire      clk,
    input  wire      rst,

    input  wire [9:0] hcount,
    input  wire [9:0] vcount,
    input  wire      active,

    input  wire [4:0] rd_data,
    output reg [18:0] rd_addr,

    input  wire [3:0] sw,

    output reg [3:0] vga_r,
    output reg [3:0] vga_g,
    output reg [3:0] vga_b
);

    wire [8:0] fb_x;
    wire [7:0] fb_y;
    assign fb_x = hcount[9:0];

```

```

assign fb_y = vcount[9:0];

wire [18:0] addr_next;
assign addr_next = (vcount * 19'd640) + hcount;

reg active_d;

always @( posedge clk or posedge rst ) begin
    if (rst) begin
        rd_addr  <= 17'd0;
        active_d <= 1'b0;
    end else begin
        if (active) rd_addr <= addr_next;
        active_d <= active;
    end
end

wire [11:0] pixel_display;

filter_engine filter (
    .pixel_in(rd_data),
    .sw(sw),
    .pixel_out(pixel_display)
);

always @( posedge clk or posedge rst ) begin
    if (rst) begin
        vga_r <= 4'd0;
        vga_g <= 4'd0;
        vga_b <= 4'd0;
    end else begin
        if (active_d) begin
            vga_r <= pixel_display[11:8];
            vga_g <= pixel_display[7:4];
            vga_b <= pixel_display[3:0];
        end else begin
            vga_r <= 4'd0;
            vga_g <= 4'd0;
            vga_b <= 4'd0;
        end
    end
end

```

```

        end
    end
end

endmodule

```

Module Description : Since the address mapping of 640 * 480 pixels per frame is different, the calculation is different too.

Module : filter_engine

Module code :

```

module filter_engine (
    input wire [4:0] pixel_in,      // input RGB221
    input wire [3:0] sw,            // Switches for filter
    output reg [11:0] pixel_out     // output RGB444
);

    wire [1:0] r2 = pixel_in[4:3];
    wire [1:0] g2 = pixel_in[2:1];
    wire b1 = pixel_in[0];

    wire [3:0] r4 = {2{r2}};
    wire [3:0] g4 = {2{g2}};
    wire [3:0] b4 = {4{b1}};

    wire [7:0] gray_sum = (r4 * 3) + (g4 * 6) + b4 + 5;
    wire [3:0] gray = gray_sum / 10;

    wire [1:0] r_invt = 3 - r2;
    wire [1:0] g_invt = 3 - g2;
    wire b_invt = !b1;

    always @(*) begin
        case (sw)
            4'b0000 : pixel_out = {r4, g4, b4};
            // Normal
            4'b0001 : pixel_out = {gray, gray, gray};

```

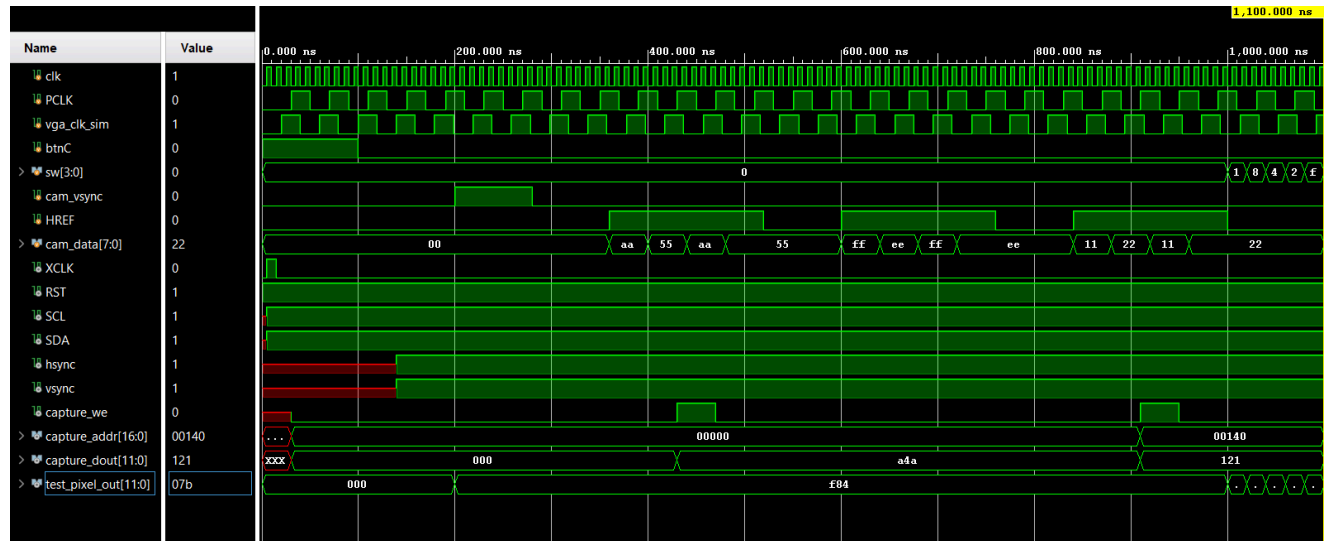
```

// Grayscale
    4'b0010 : pixel_out = {4'b0000, 4'b0000, b4};
// Blue Only
    4'b0100 : pixel_out = {4'b0000, g4, 4'b0000};
// Green Only
    4'b1000 : pixel_out = {r4, 4'b0000, 4'b0000};
// Red only
    4'b1111 : pixel_out = {{2{r_invt}}, {2{g_invt}},
{4{b_invt}}}; // Inversion
    default : pixel_out = {r4, g4, b4};
    endcase
end
endmodule

```

Module Description : Since the stored bits are 5 bits which is different from the 320 * 240 pixels per frame implementation, the filter is calculated difference.

Simulation & Testbenches



```
-----
RUNNING TEST: 320x240 | Color Mode: 12-bit | Test Pixel: 12'hf84
-----
```

```
System: XCLK is toggling [PASS]
VGA Sync: Signals are defined [PASS]
SCCB Interface: Bus signals are alive [PASS]
Capture: Check addr for 0, 1 (Line 0) and 320, 321 (Line 2)
INFO: [USF-XSim-96] XSim completed. Design snapshot 'top_module_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:13 . Memory (MB): peak = 2400.613 ; gain = 0.000
run all
Grayscale: Out=12'h666 | Exp:12'h666 [PASS]
Red Filter: Out=12'hf00 | Exp:12'hf00 [PASS]
Green Filter: Out=12'h080 | Exp:12'h080 [PASS]
Blue Filter: Out=12'h004 | Exp:12'h004 [PASS]
Inversion: Out=12'h07b | Exp:12'h07b [PASS]
-----
```

```
Simulation finished at 1100000
```

code :

```
`timescale 1ns / 1ps

module top_module_tb;
    // --- Configuration ---
    parameter MODE = 1; // 0 = RGB444, 1 = RGB221

    localparam H_ACTIVE = (MODE) ? 640 : 320;
    localparam V_ACTIVE = (MODE) ? 480 : 240;
    localparam PIXEL_BITS = (MODE) ? 5 : 12;
```

```

// --- Inputs ---
reg clk;
reg btnC;
reg [3:0] sw;
reg PCLK;
reg cam_vsync;
reg HREF;
reg [7:0] cam_data;

// --- Outputs ---
wire init_ready;
wire XCLK, RST, PWDN, SCL;
wire SDA;
wire hsync, vsync;
wire [3:0] vga_r, vga_g, vga_b;

// --- Instantiate UUT ---
top_module uut (
    .clk(clk),
    .btnC(btnC),
    .init_ready(init_ready),
    .sw(sw),
    .PCLK(PCLK),
    .cam_vsync(cam_vsync),
    .HREF(HREF),
    .cam_data(cam_data),
    .XCLK(XCLK),
    .RST(RST),
    .PWDN(PWDN),
    .SCL(SCL),
    .SDA(SDA),
    .hsync(hsync),
    .vsync(vsync),
    .vga_r(vga_r),
    .vga_g(vga_g),
    .vga_b(vga_b)
);

// --- Instantiate Capture Module
wire capture_we;
wire [16:0] capture_addr;
wire [11:0] capture_dout;
ov7670_capture test_capture (
    .pclk(PCLK),
    .rst(btnC),

```



```

        .vsync(cam_vsync),
        .href(HREF),
        .d(cam_data),

        .we(capture_we),
        .addr(capture_addr),
        .dout(capture_dout),
        .frame_done()
    );

    // --- Instantiate filter engine---
    reg [PIXEL_BITS-1:0] test_pixel;
    wire [11:0] test_pixel_out;
    filter_engine test_filter (
        .pixel_in(test_pixel),
        .sw(sw),
        .pixel_out(test_pixel_out)
    );

    // --- Clock generation ---
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 1 T = 10ns
    end

    initial begin
        PCLK = 0;
        #10;
        forever #20 PCLK = ~PCLK; // 1 T = 40ns
    end

    reg vga_clk_sim;
    initial begin
        vga_clk_sim = 0;
        forever #19.86 vga_clk_sim = ~vga_clk_sim; // 25.175 MHz (T = 39.72ns)
    end

    // --- Main Test ---
    reg [11:0] expected_val;

    // for caculate grayscale
    wire [3:0] r4_tb = (MODE) ? {2{test_pixel[4:3]}} : test_pixel[11:8];
    wire [3:0] g4_tb = (MODE) ? {2{test_pixel[2:1]}} : test_pixel[7:4];
    wire [3:0] b4_tb = (MODE) ? {4{test_pixel[0]}} : test_pixel[3:0];

```

```

wire [5:0] gray_sum_m0 = r4_tb + g4_tb + b4_tb;
wire [3:0] gray_m0     = gray_sum_m0[5:2];

wire [7:0] gray_sum_m1 = (r4_tb * 3) + (g4_tb * 6) + b4_tb + 5;
wire [3:0] gray_m1     = gray_sum_m1 / 10;

wire [3:0] gray_tb = (MODE) ? gray_m1 : gray_m0;

initial begin
    force uut.vga_timing.clk = vga_clk_sim; // clock for VGA

    // Initialize
    btnC = 1;
    sw = 4'b0000;
    cam_vsync = 0;
    HREF = 0;
    cam_data = 8'h00;
    test_pixel = 12'h000;
    #100;
    btnC = 0;
    #100;

    if (MODE) test_pixel = 5'b11101;
    else test_pixel = 12'hF84;

$display("-----");
    if (MODE) begin
        $display("RUNNING TEST: %0dx%0d | Color Mode: %0d-bit | Test
Pixel: 5'b%b", H_ACTIVE, V_ACTIVE, PIXEL_BITS, test_pixel);
    end else begin
        $display("RUNNING TEST: %0dx%0d | Color Mode: %0d-bit | Test
Pixel: 12'h%h", H_ACTIVE, V_ACTIVE, PIXEL_BITS, test_pixel);
    end

$display("-----");

    // Test system & clock
    if (XCLK != 1'bx) $display("System: XCLK is toggling [PASS]");
    else $display("System: XCLK is undefined [FAIL]");

    // --- Test VGA Sync ---
    if (hsync != 1'bx && vsync != 1'bx) $display("VGA Sync: Signals are
defined [PASS]");
    else $display("VGA Sync: Signals are X (Undefined) [FAIL]");

```

```

// --- Test SCCB ---
if (SCL != 1'bx && SDA != 1'bx) $display("SCCB Interface: Bus
signals are alive [PASS]");
else $display("SCCB Interface: Bus signals are X [FAIL]");

// Test capture logic & memory addressing
cam_vsync = 1; #80;
cam_vsync = 0; #80;

// Line 0: stored data
HREF = 1;
repeat(2) begin
    cam_data = 8'hAA; #40;
    cam_data = 8'h55; #40;
end
HREF = 0; #80;

// Line 1: skip
HREF = 1;
repeat(2) begin
    cam_data = 8'hFF; #40;
    cam_data = 8'hEE; #40;
end
HREF = 0; #80;

// Line 2: stored data
HREF = 1;
repeat(2) begin
    cam_data = 8'h11; #40;
    cam_data = 8'h22; #40;
end
HREF = 0;
$display("Capture: Check addr for 0, 1 (Line 0) and 320, 321 (Line
2)");

// --- Test grayscale ---
sw = 4'b0001; #20;
expected_val = {gray_tb,gray_tb,gray_tb};
$display("Grayscale: Out=%h | Exp:%h [%s]", test_pixel_out,
expected_val, (test_pixel_out == expected_val ? "PASS" : "FAIL"));

// Test red chananel
sw = 4'b1000; #20;
expected_val = (MODE) ? { {2{test_pixel[4:3]}} , 8'h00 } : 12'hF00;

```

```

        $display("Red Filter: Out=%h | Exp:%h [%s]", test_pixel_out,
expected_val, (test_pixel_out == expected_val ? "PASS" : "FAIL"));

        // Test green channel
        sw = 4'b0100; #20;
        expected_val = (MODE) ? { 4'h0, {2{test_pixel[2:1]}}, 4'h0 } :
12'h080;
        $display("Green Filter: %h | Exp:%h [%s]", test_pixel_out,
expected_val, (test_pixel_out == expected_val ? "PASS" : "FAIL"));

        // Test blue channel
        sw = 4'b0010; #20;
        expected_val = (MODE) ? { 8'h00, {4{test_pixel[0]}} } : 12'h004;
        $display("Blue Filter: %h | Exp:%h [%s]", test_pixel_out,
expected_val, (test_pixel_out == expected_val ? "PASS" : "FAIL"));

        // Test color inversion
        sw = 4'b1111; #20;
        if (MODE) begin
            expected_val = { {2{2'd3-test_pixel[4:3]}},
{2{2'd3-test_pixel[2:1]}}, {4{!test_pixel[0]}} };
        end else begin
            expected_val = ~test_pixel;
        end
        $display("Color Inversion: %h | Exp:%h [%s]", test_pixel_out,
expected_val, (test_pixel_out == expected_val ? "PASS" : "FAIL"));
        sw = 4'b0000;

$display("-----");
        $display("Simulation finished at %t", $time);

        release uut.vga_timing.clk;

        $finish;

    end

endmodule

```

Challenge Faced

1. There is a lot of documentation to read.
2. It's hard to implement code from scratch.
3. It spends too much time debugging color bits and handling clock domain crossing.
4. There are many specialized words that are difficult to understand at first read.

AI Usage

1. Using for guiding overall workflow
2. Using for debugging code
3. Find reference and research
4. Generate Testbranch to test module
5. Generate configuration setting of camera